

Biểu diễn bài toán và tìm kiếm

Problem formulation and search

Phạm Minh Tuấn, PhD

Nội dung

- Tổng quan về tìm kiếm
- Biểu diễn bài toán trong không gian trạng thái
- Đồ thị tìm kiếm
- Cây tìm kiếm
- Tìm kiếm không có thông tin
 - Tìm kiếm rộng
 - Tìm kiếm sâu

Tổng quan

- ✓ **Tìm kiếm** là thuật toán lấy đầu vào là một bài toán và trả về kết quả là một lời giải cho bài toán đó, thường là sau khi cân nhắc giữa một loạt các lời giải có thể.
- ✓ **Không gian trạng thái** trong tìm kiếm
 - ✓ Là tập các trạng thái có thể có của bài toán và các toán tử di chuyển trạng thái
 - ✓ Được biểu diễn bằng cây, đồ thị,...
- ✓ Các phương pháp tìm kiếm
 - ✓ Tìm kiếm không có thông tin
 - ✓ Tìm kiếm có thông tin
 - ✓ Tìm kiếm đối kháng

Biểu diễn trong không gian trạng thái

Trò chơi 8 số

- ✓ Trạng thái đầu: {1, null, 4, 6, 3, 2, 7, 8, 5}
- ✓ Trạng thái cuối: {1, 2, 3, 4, 5, 6, 7, 8, null}
- ✓ Toán tử:
 - ✓ **Up:** Ô ở dưới ô trống di chuyển lên phía trên
 - ✓ **Down:** Ô ở trên ô trống di chuyển xuống phía dưới
 - ✓ **Left:** Ô ở bên phải ô trống di chuyển sang trái
 - ✓ **Right:** Ô ở bên trái ô trống di chuyển sang phải
- ✓ Trạng thái sẽ thay đổi phụ thuộc và toán tử
- ✓ Tùy thuộc vào trạng thái mà sẽ có toán tử không thực hiện được

Trạng thái đầu

1		4
6	3	2
7	8	5

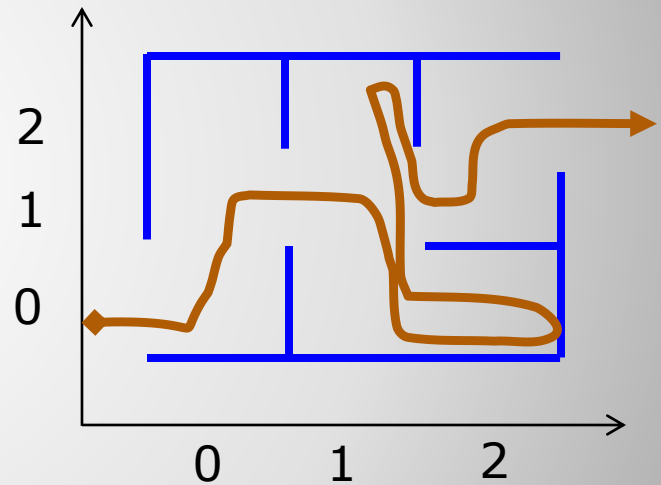
Trạng thái cuối

1	2	3
4	5	6
7	8	

Biểu diễn trong không gian trạng thái

Tìm đường trong Mê cung

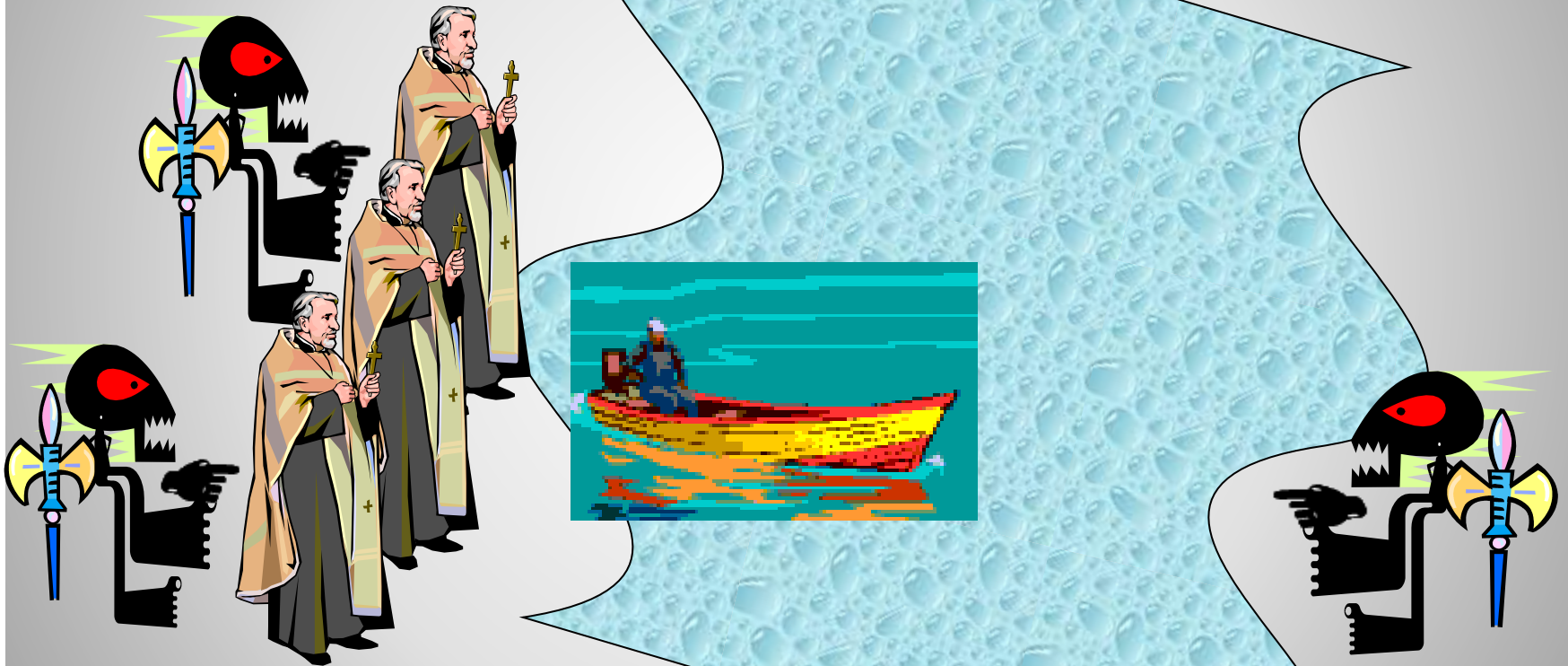
- ✓ Trạng thái đầu: $\{0,0\}$
- ✓ Trạng thái cuối: $\{2,2\}$
- ✓ Toán tử:
 - ✓ **Up:** Di chuyển lên trên
 - ✓ **Down:** Di chuyển xuống dưới
 - ✓ **Left:** Di chuyển sang trái
 - ✓ **Right:** Di chuyển sang phải
- ✓ Trạng thái sẽ thay đổi phụ thuộc và toán tử
- ✓ Tùy thuộc vào trạng thái mà sẽ có toán tử không thực hiện được



Biểu diễn trong không gian trạng thái

Trạng thái đầu: {3,3, trái}

Trạng thái cuối: {0,0, phải}



Trạng thái: {3,2, trái}

Số đạo sĩ ở bên trái

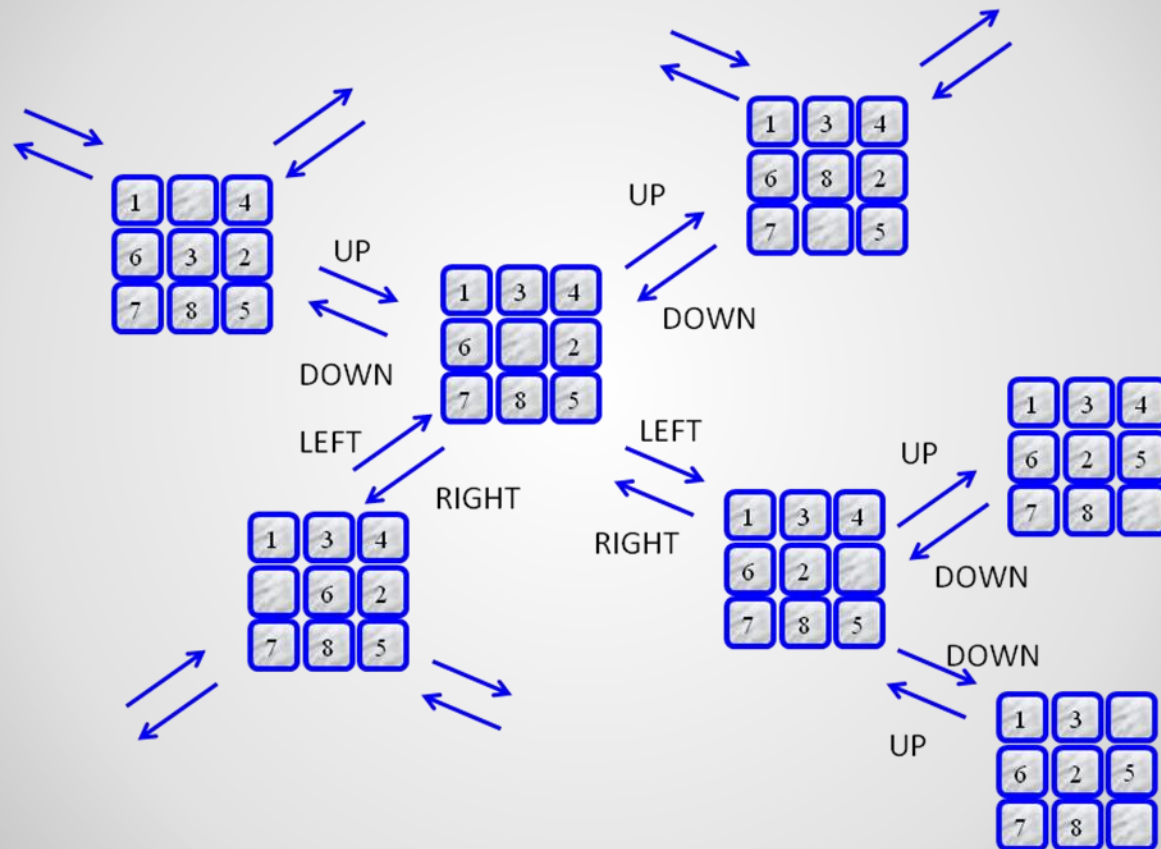
Số con quỷ ở bên trái

Vị trí thuyền

Các bài toán khác

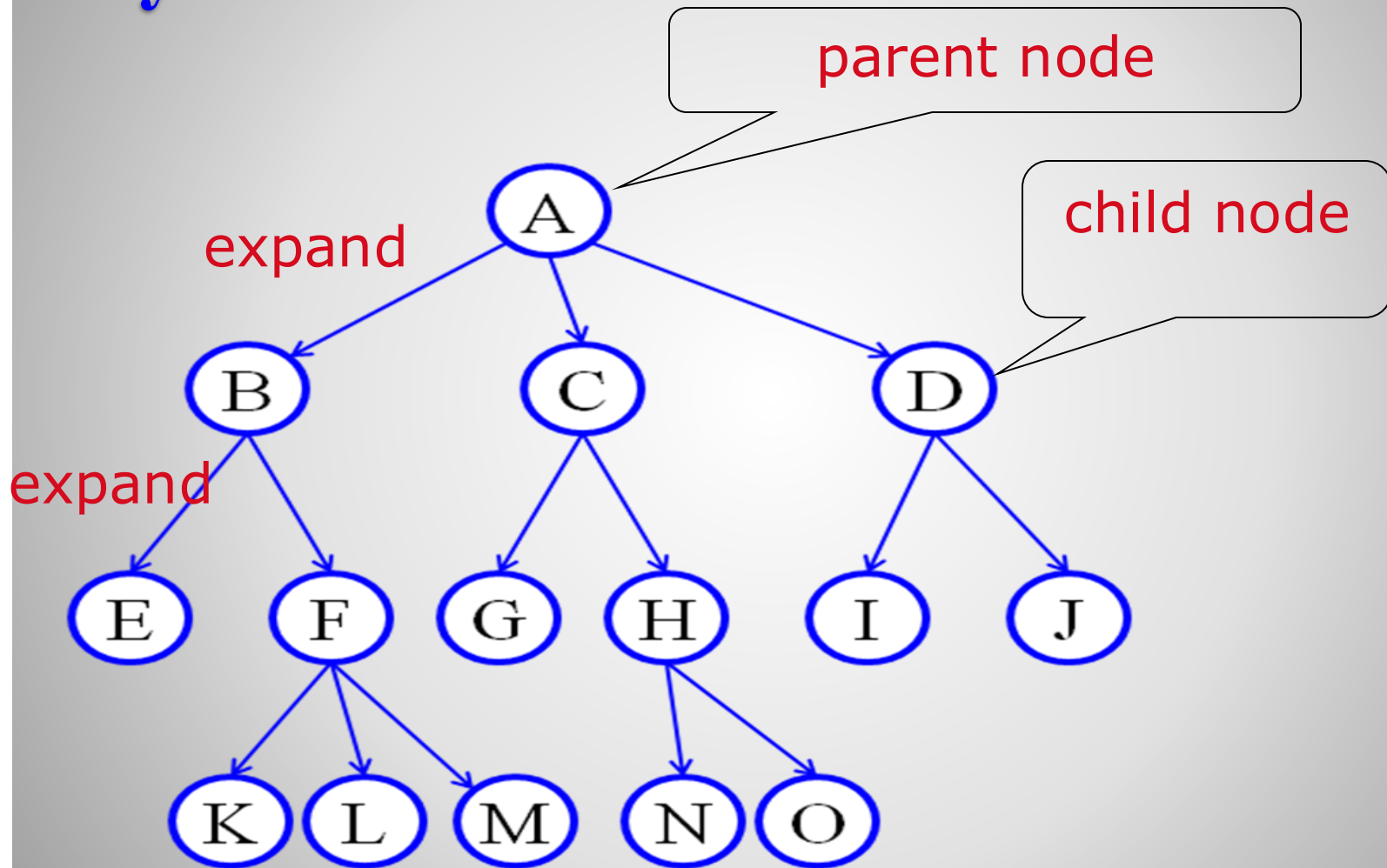
- ✓ Bài toán tháp hanoi
- ✓ Bài toán 8 con hậu
- ✓ Cờ vua, cờ tướng
- ✓ Tìm đường
- ✓ Tìm kiếm thông tin trên internet
- ✓ Game AI (PES, Mario,...)

Đồ thị tìm kiếm



Đồ thị có hướng của trò chơi 8 số

Cây Tìm kiếm

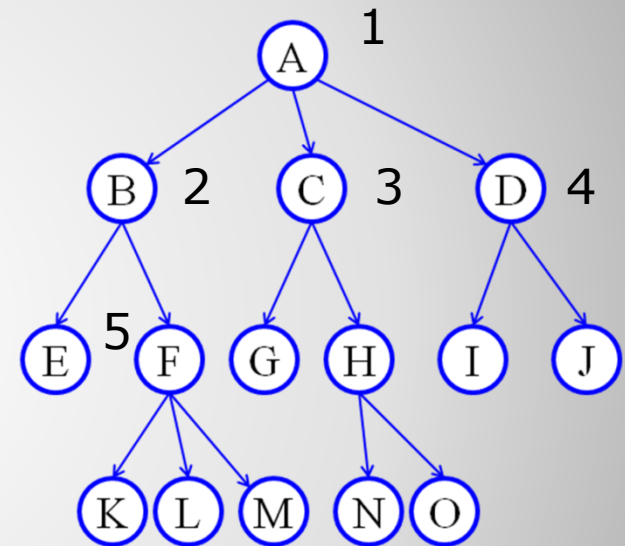


Tìm kiếm không có thông tin

- ✓ Tìm kiếm theo chiều rộng
- ✓ Tìm kiếm theo chiều sâu
- ✓ Tìm kiếm sâu dần

Tìm kiếm rộng

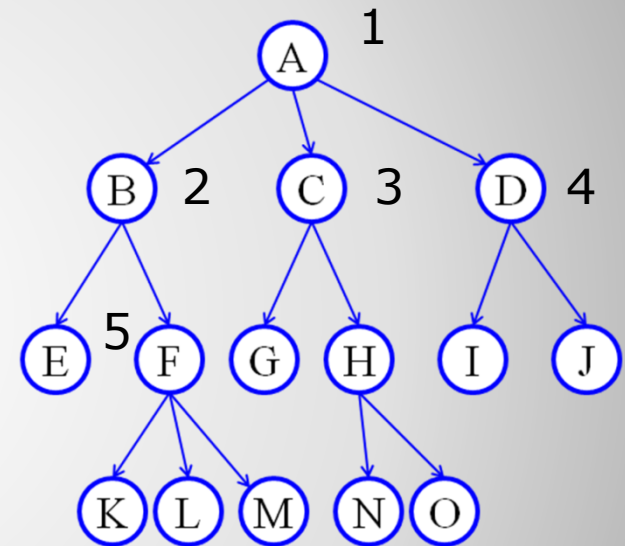
- ✓ Tìm kiếm rộng là phương pháp tìm kiếm mà ưu tiên những đỉnh gần với đỉnh xuất phát.
- ✓ Thứ tự tìm kiếm: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow K \rightarrow L \rightarrow M \rightarrow N \rightarrow O$.



Tìm kiếm rộng

Thuật toán tìm kiếm rộng:

1. Cho đỉnh xuất phát vào **open**.
2. Nếu **open** rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh **đầu** trong **open** ra và gọi đó là Θ .
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của Θ và cho vào **cuối** của **open**
6. Trở lại bước 2.



*Open hoạt động theo quy tắc FIFO

Tìm kiếm rộng – Ví dụ 1

Quá trình biến đổi của open:

open= {A}

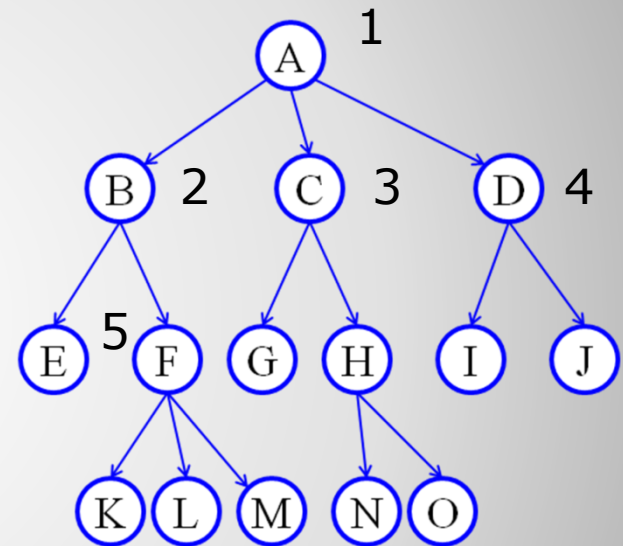
open= {**B**, C, D}

open= {C, D, E, F}

open= { D, E, F, **G**, **H** }

open= { E, F, G, H, I, J}

...

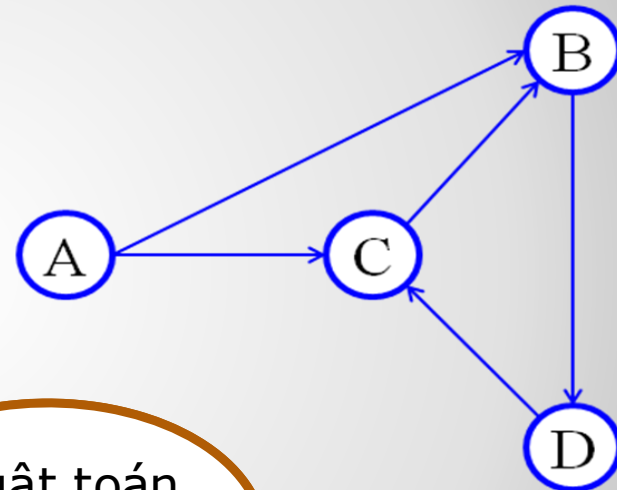


Tìm kiếm rộng - Ví dụ 2

Quá trình biến đổi của open:

open = {A}
open = {**B**, **C**}
open = {C, **D**}
open = {D, **B**}
open = {**B**, **C**}
open = {C, **D**}
...

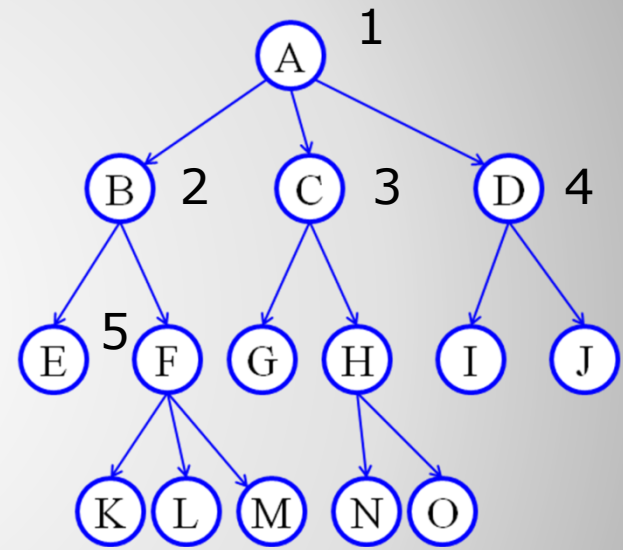
Thuật toán
không kết
thúc???



Tìm kiếm rộng – Thuật toán 2

Thuật toán tìm kiếm rộng:

1. Cho đỉnh xuất phát vào open.
2. Nếu open rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh đầu trong open ra và gọi đó là Θ . **Cho Θ vào closed**
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của **Θ không thuộc open và closed** cho vào cuối của open
6. Trở lại bước 2.



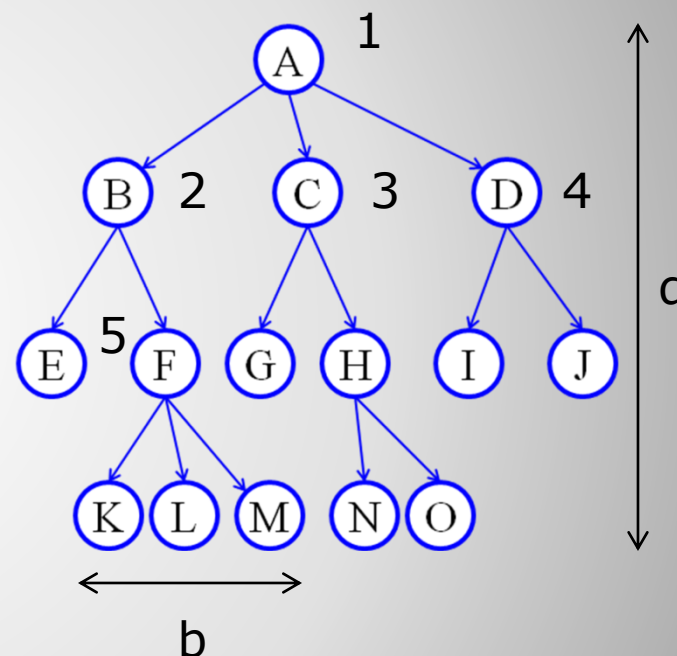
Tìm kiếm rộng – Đánh giá thuật toán

Tổng số đỉnh cần xét tối đa là:
 $1 + b + b^2 + \dots + b^d$.

Trong đó:

b: số nhánh tối đa của 1 trạng thái

d: độ sâu

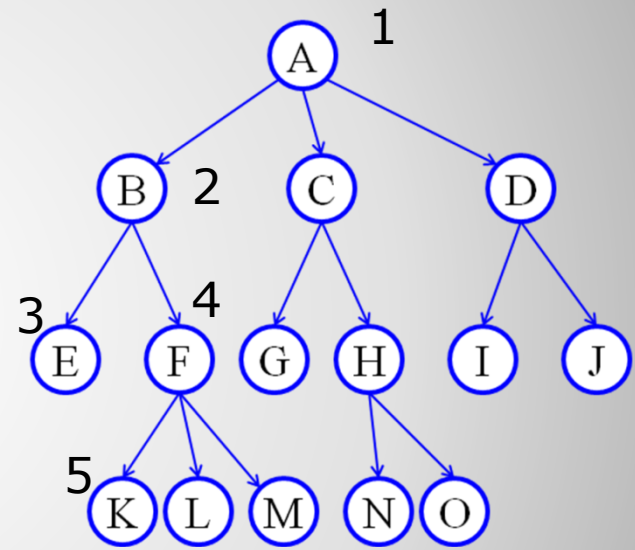


Như vậy độ phức tạp về thời gian và không gian (bộ nhớ) của thuật toán là $O(b^d)$.

Tìm kiếm sâu

✓ Tìm kiếm sâu là phương pháp tìm kiếm mà ưu tiên tìm kiếm những đỉnh xa với đỉnh xuất phát.

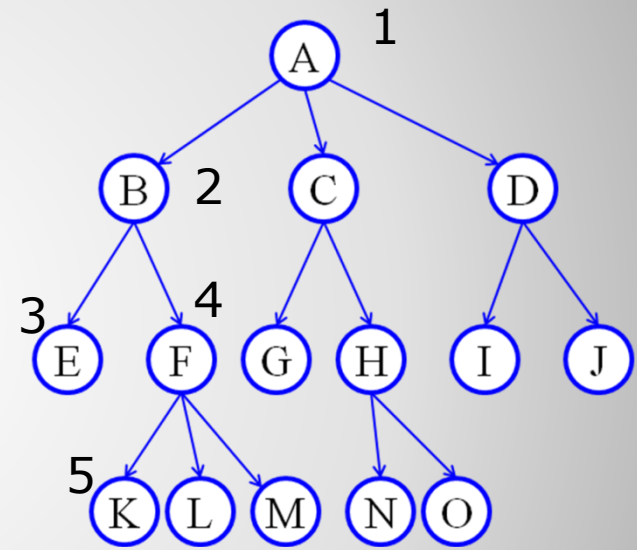
✓ Thứ tự tìm kiếm: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow K \rightarrow L \rightarrow M \rightarrow C \rightarrow G \rightarrow H \rightarrow N \rightarrow O \rightarrow D \rightarrow I \rightarrow J$.



Tìm kiếm sâu

Thuật toán tìm kiếm sâu:

1. Cho đỉnh xuất phát vào open.
2. Nếu open rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh đầu trong open ra và gọi đó là Θ . Cho Θ vào closed
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của Θ không thuộc open và closed **cho vào đầu của open**
6. Trở lại bước 2.



*Open hoạt động theo quy tắc LIFO

Tìm kiếm sâu – Ví dụ

Quá trình biến đổi của open:

open= {A}

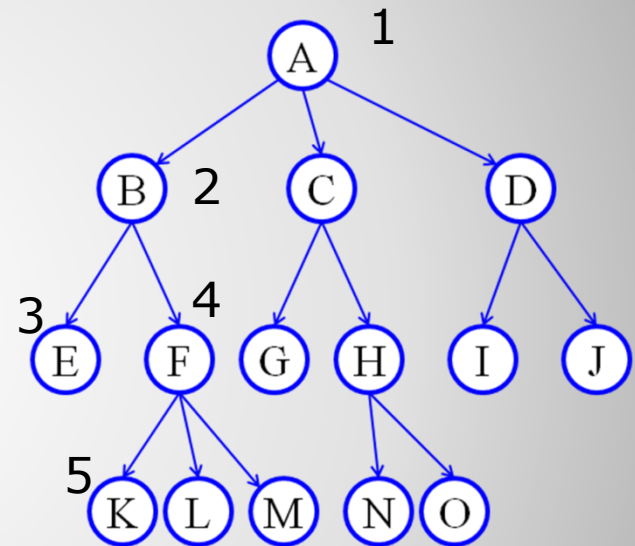
open= {B, C, D}

open= {E, F, C, D}

open= {F, C, D}

open= {K, L, M, C, D}

...



Tìm kiếm sâu – Đánh giá thuật toán

Tổng số đỉnh cần xét tối đa là:

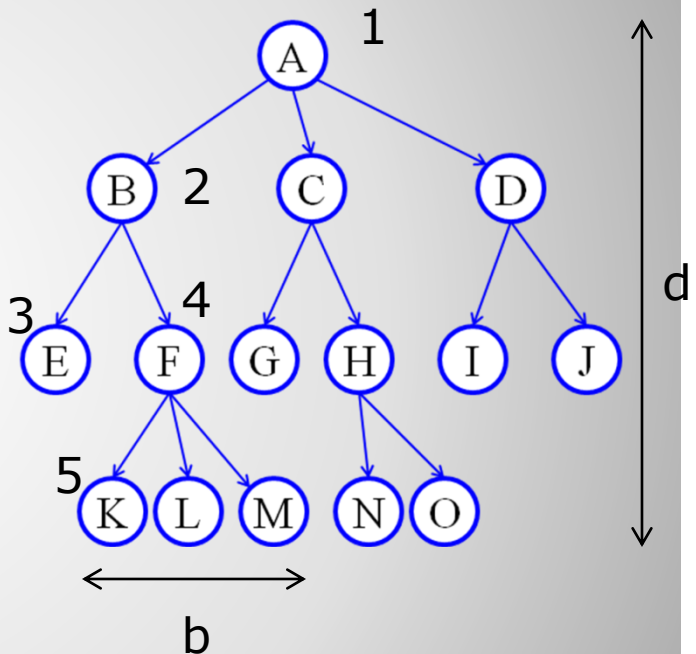
$$1 + b + b^2 + \dots + b^d.$$

Tổng số đỉnh tối đa trong open là: $3 \times b^d$

Trong đó:

b: số nhánh tối đa của 1 trạng thái

d: độ sâu



Như vậy độ phức tạp về thời gian của thuật toán là $O(b^d)$ và không gian (bộ nhớ) của thuật toán là $O(bd)$.

Kết luận

- Giới thiệu một số bài toán và cách biểu diễn bài toán trong không gian trạng thái
- Biểu diễn trên
 - Đồ thị tìm kiếm
 - Cây tìm kiếm
- Tìm kiếm không có thông tin
 - Tìm kiếm rộng
 - Tìm kiếm sâu

Tìm kiếm có thông tin

- ✓ Tìm kiếm Heuristic
 - ✓ Tìm kiếm chi phí cực tiểu (Uniform cost search)
 - ✓ Tìm kiếm ước lượng (Best-first search)
 - ✓ A* algorithm
- ✓ Kiểm tra trên cây trò chơi (Game tree search)

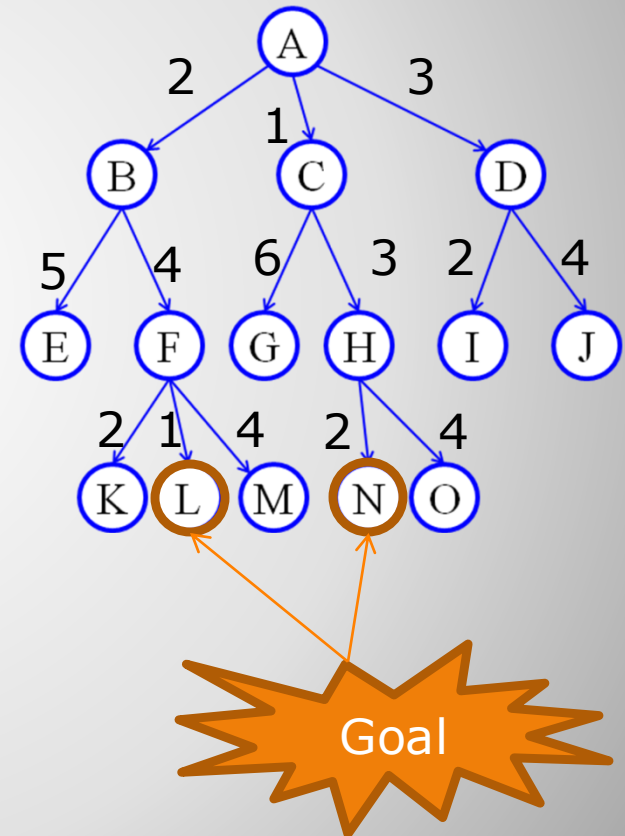
Uniform cost search

- ✓ Đặt vấn đề
 - ✓ Khi một bài toán có nhiều nghiệm thì thường chọn những phương án có “**chi phí thấp nhất.**”
- ✓ Ví dụ:
 - ✓ Tìm đường đi từ TP. A đến TP. B sao cho:
 - ✓ Nhanh nhất
 - ✓ Rẻ nhất
- ✓ Các phương pháp tìm kiếm sâu và tìm kiếm rộng không đảm bảo việc tối ưu cho bài toán “**chi phí thấp nhất.**”
- ✓ Uniform cost search
 - ✓ Đảm bảo tìm được “**chi phí thấp nhất.**”

Uniform cost search

Thuật toán tìm kiếm chi phí cực tiểu:

1. Cho đỉnh xuất phát vào open.
2. Nếu open rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh đầu trong open ra và gọi đó là Θ . Cho Θ vào closed
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của Θ không thuộc open và closed cho vào open **theo thứ tự tăng dần về khoảng cách từ đỉnh xuất phát**.
6. Trở lại bước 2.



Uniform cost search – Ví dụ

Quá trình biến đổi của open:

open= {A(0)}

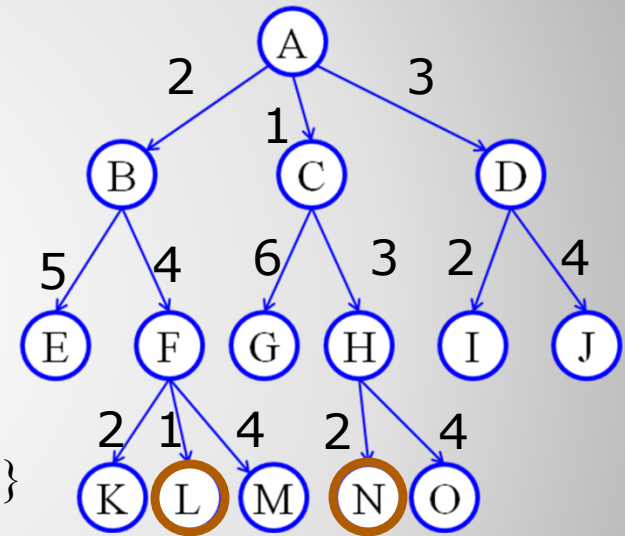
open= {C(1), B(2), D(3)}

open= {B(2), D(3), H(1+3=4), G(1+6=7)}

open= {D(3), H(4), F(2+4=6), G(7), E(2+5=7)}

open= {H(4), I(3+2=5), F(6), G(7), E(7), J(3+4=7)}

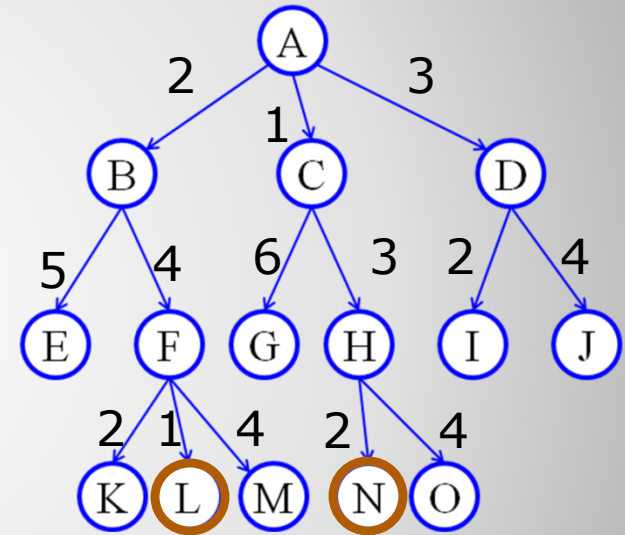
...



(Bài tập: Tiếp tục quá trình trên đến khi thuật toán kết thúc)

Uniform cost search – Tính chất

- ✓ Lời giải được phát hiện đầu tiên cũng là nghiệm có “chi phí thấp nhất”
- ✓ Nếu bài toán có lời giải, thì đảm bảo thuật toán sẽ dừng
- ✓ Tất cả các nhánh có chi phí là 1 thì thuật toán trở thành tìm kiếm rộng



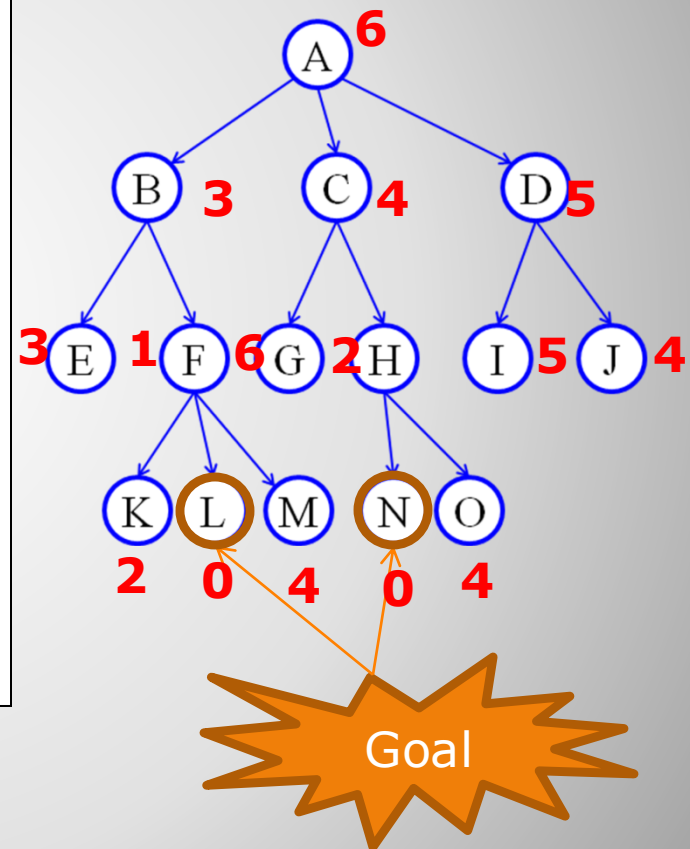
Tại sao lại tìm kiếm Heuristic?

- ✓ Heuristic là gì?
 - ✓ Theo kinh nghiệm
- ✓ Ví dụ:
 - ✓ Khi không rõ đường đi, thường thì mọi người nhắm về hướng cần tới để đi. (Ưu tiên đường đi mà có khoảng cách đường chim bay ngắn nhất)
- ✓ Tìm kiếm Heuristic là tìm kiếm sử dụng kiến thức phỏng đoán chi phí từ đỉnh đang xét đến đích.
- ✓ Thuật toán

Best-first search

Thuật toán tìm kiếm ước lượng:

1. Cho đỉnh xuất phát vào open.
2. Nếu open rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh đầu trong open ra và gọi đó là Θ . Cho Θ vào closed
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của Θ không thuộc open và closed cho vào open **theo thứ tự tăng dần về khoảng cách ước lượng đến đích**.
6. Trở lại bước 2.



Best-first search – Ví dụ

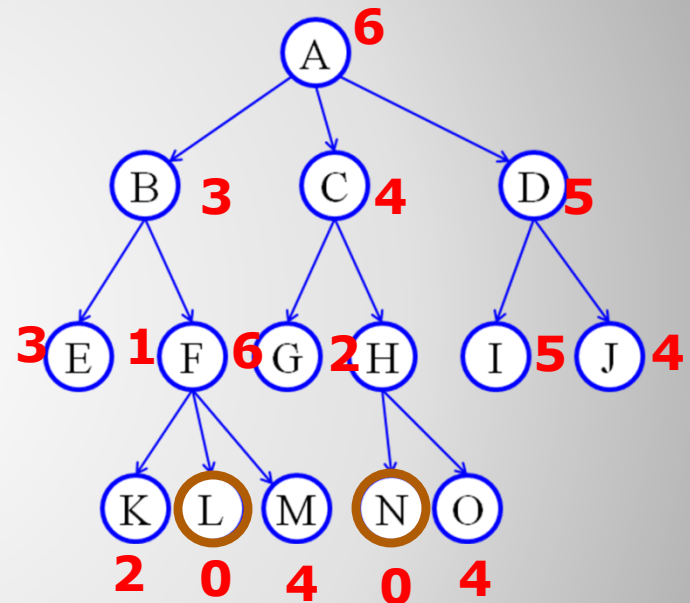
Quá trình biến đổi của open:

open= {A(6)}

open= {B(3), C(4), D(5)}

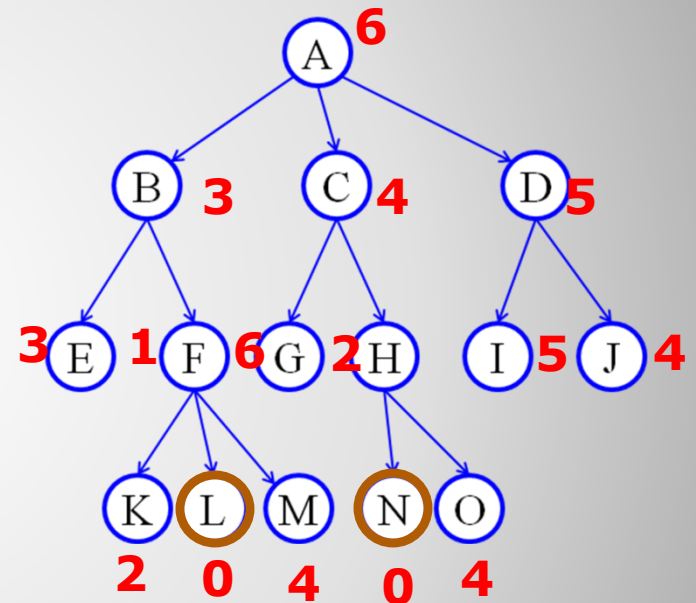
open= {F(1), E(3), C(4), D(5)}

open= {L(0), K(2), E(3), C(4), M(4), D(5)}



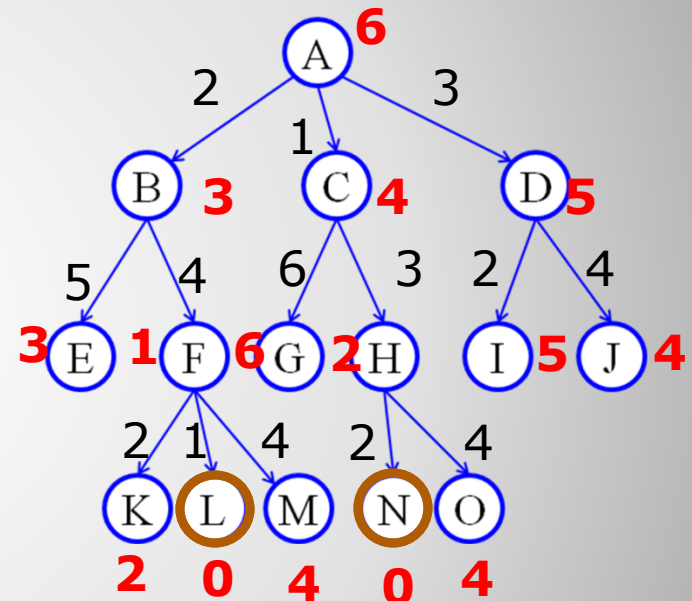
Best-first search – Tính chất

- ✓ Lời giải được phát hiện đầu tiên **không đảm bảo** “chi phí thấp nhất”
- ✓ So với **Uniform cost search** thì tiết kiệm về mặt không gian hơn



Thuật toán A* (A* Algorithm)

- ✓ Thuật toán A* kết hợp **Uniform cost search** và **Best-first search** nhằm tăng hiệu quả tìm kiếm
- ✓ Sử dụng khoảng cách từ đỉnh xuất phát đến đỉnh đang xét: $g(X)$
- ✓ Sử dụng khoảng cách ước lượng từ đỉnh đang xét đến đích $h'(X)$



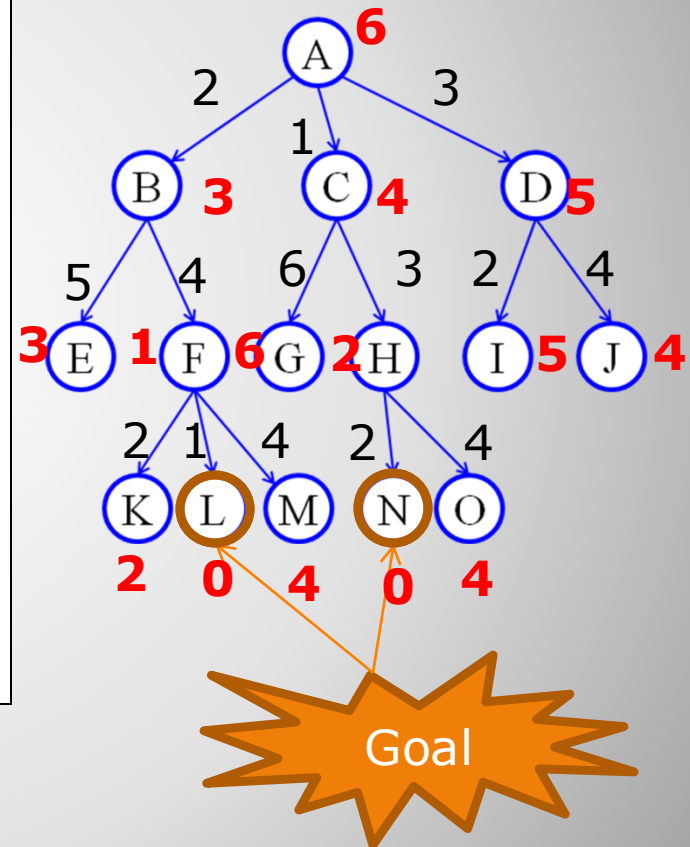
Thuật toán A* (A* Algorithm)

Thuật toán A*:

1. Cho đỉnh xuất phát vào open.
2. Nếu open rỗng thì tìm kiếm thất bại, kết thúc việc tìm kiếm.
3. Lấy đỉnh đầu trong open ra và gọi đó là Θ . Cho Θ vào closed
4. Nếu Θ là đỉnh đích thì tìm kiếm thành công, kết thúc việc tìm kiếm.
5. Tìm tất cả các đỉnh con của Θ không thuộc open và closed cho vào open **theo thứ tự tăng dần đối với hàm $f(X)=g(X)+h'(X)$** .
6. Trở lại bước 2.

$g(X)$: Khoảng cách từ đỉnh xuất phát đến X

$h'(X)$: Khoảng cách ước lượng từ X đến đích



Thuật toán A* (A* Algorithm) – Ví dụ

Quá trình biến đổi của open:

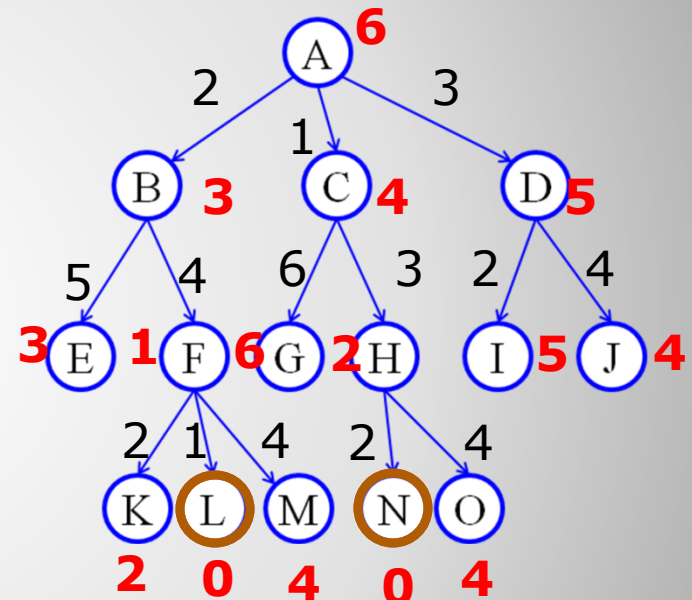
open= {A(0+6)}

open= {B(2+3), C(1+4), D(3+5)}

open= {C(1+4), F(6+1), D(3+5), E(7+3)}

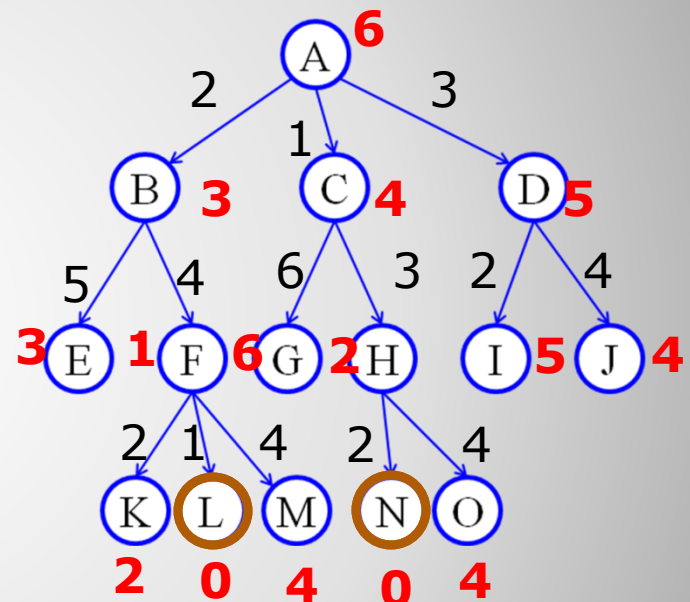
open= {H(4+2), F(6+1), D(3+5), E(7+3), G(7+6)}

open= {N(6+0), F(6+1), D(3+5), E(7+3), O(8+4), G(7+6)}



Thuật toán A* – Tính chất

- ✓ Lời giải được phát hiện đầu tiên **đảm bảo** “chi phí thấp nhất” khi tất cả khoảng cách ước lượng từ đỉnh đang xét đến đích nhỏ hơn hoặc bằng khoảng cách thực. (*)
- ✓ So với **Uniform cost search** thì tiết kiệm về mặt không gian
- ✓ Nhanh hơn so với thuật toán khác (**Từ kết quả thực nghiệm**)



✓ BT: Chứng minh (*)?

- ✓ Gợi ý: Hãy chứng minh với mọi x là các đỉnh thuộc đường đi có chi phí thấp nhất thì $f(x) < f(G)$, G là đỉnh đích không phải chi phí thấp nhất.

Kiểm trên cây trò chơi

Phạm Minh Tuấn, PhD

Kiểm trên cây trò chơi (Game tree search)

- ✓ Đặt vấn đề
 - ✓ Thuật toán A^* chỉ thích hợp với các bài toán không có tính đối kháng
 - ✓ VD: Puzzle, Mê cung, n con Hâu
- ✓ Các bài toán có tính đối kháng như: Caro, cờ vua,...thì mọi người chơi đều muốn mình đến được đích và **cản phá đối phương không tới được đích.** → thuật toán A^* không phát huy được tác dụng

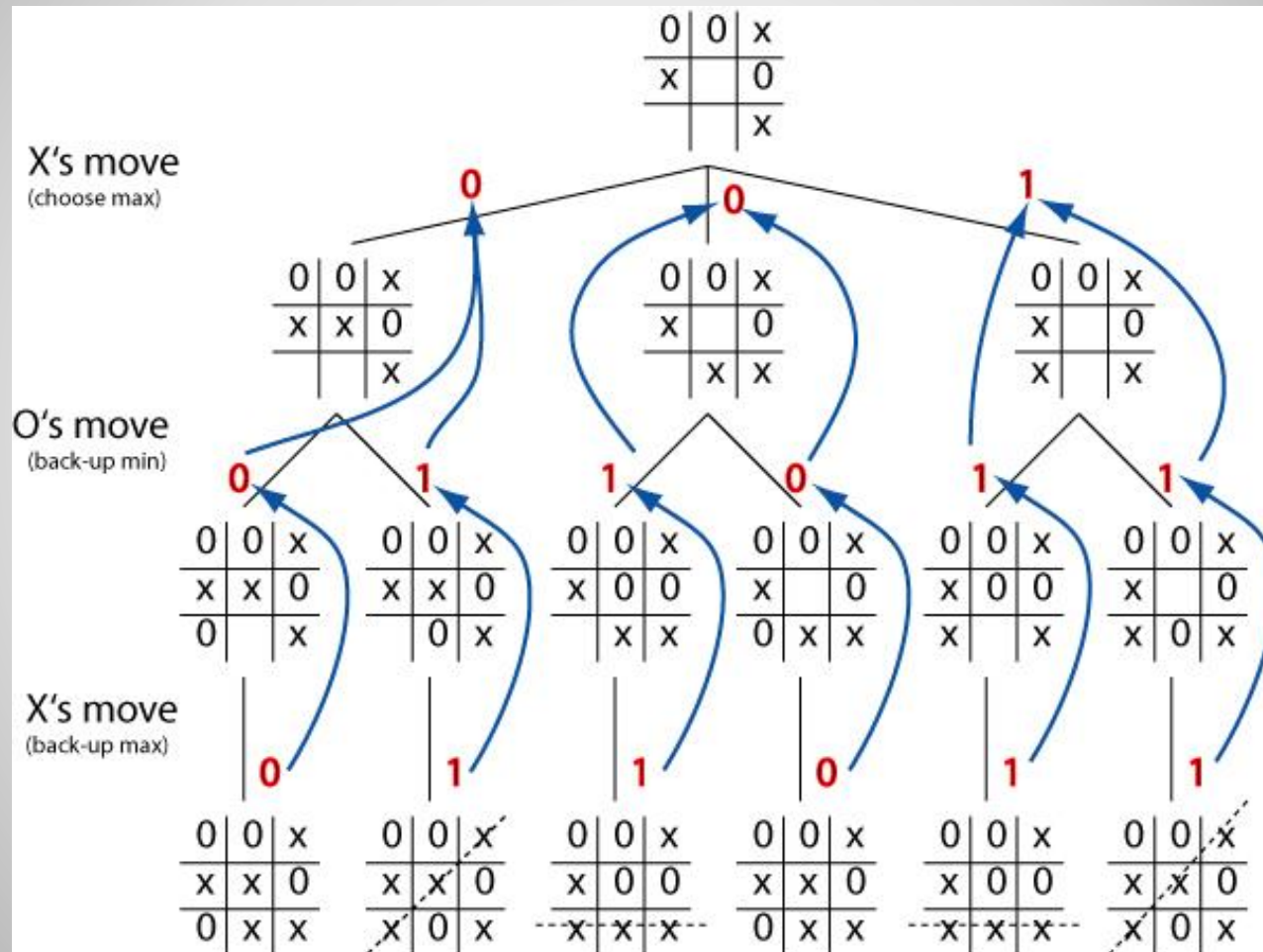
Giải thuật MiniMax

- ✓ Giải thuật MiniMax được xây dựng trên 2 giả thiết:
 - ✓ Cả 2 đối thủ có cùng kiến thức và không gian trạng thái của trò chơi
 - ✓ Cả 2 đối thủ có cùng mức cố gắng như nhau.

Giải thuật MiniMax

- ✓ Trong trò chơi đối kháng 2 người
 - ✓ Mỗi bên thay phiên nhau chơi
 - ✓ Bên địch: Làm nhỏ tối đa lợi thế của ta
 - ✓ Bên ta: Làm lớn tối đa lợi thế của ta
- ✓ Cây trò chơi
 - ✓ Phân thành các lớp Min-Max xen kẽ nhau
 - ✓ Lớp Min: lấy giá trị min của các node con
 - ✓ Lớp Max: lấy giá trị max của các node con

Giải thuật MiniMax - Ví dụ - Tic-tac-toe



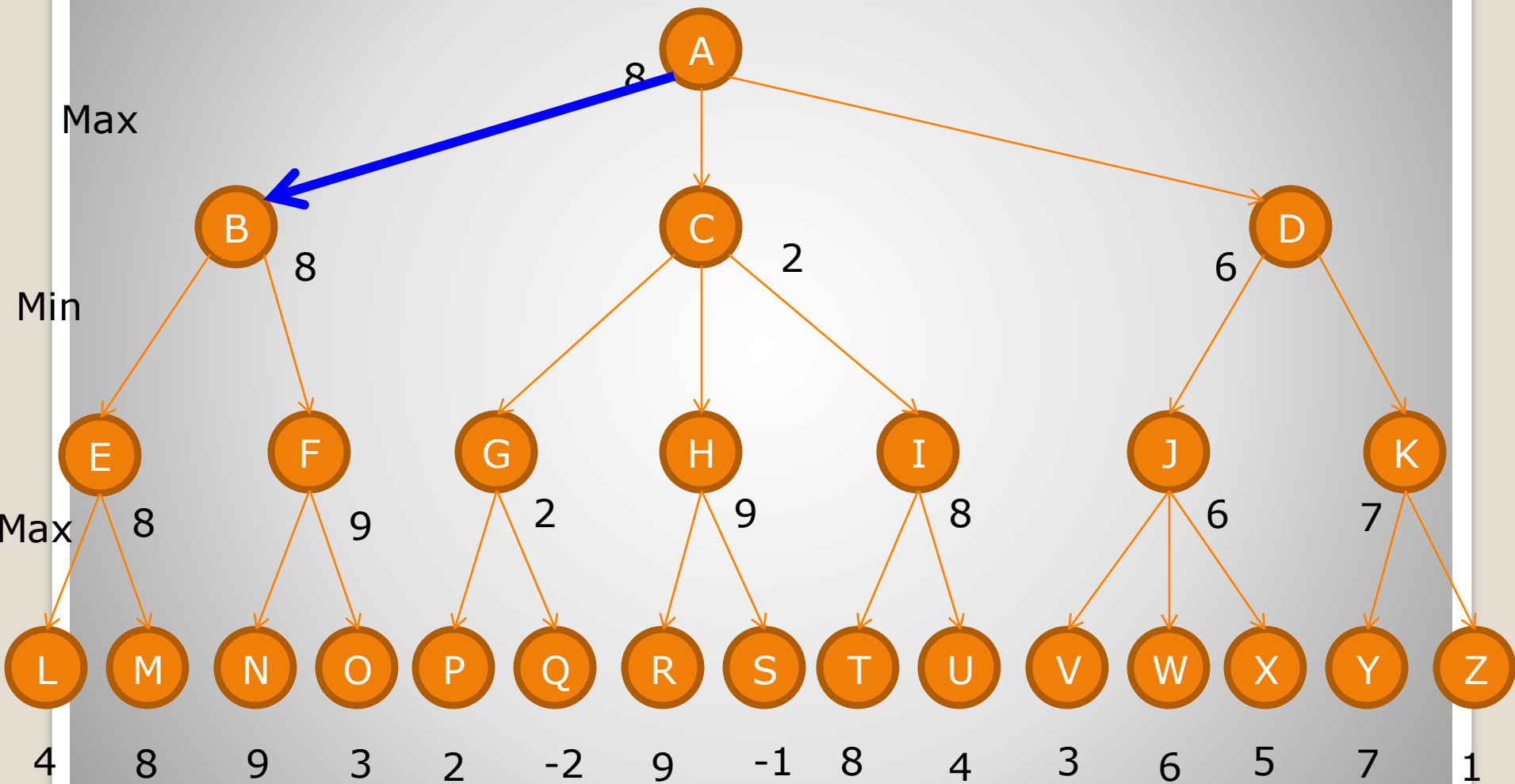
MiniMax với độ sâu giới hạn

- ✓ Minimax như đã xét buộc phải có toàn bộ không gian trạng thái đã được triển khai để có thể gán trị cho các nút lá và tính ngược lại
→ Không khả thi vì không gian bài toán lớn
- ✓ Hướng giải quyết: **Giới hạn** không gian trạng thái theo một **độ sâu** nào đó và giới hạn các **node con** theo một quy tắc nào đó

Thuật toán MiniMax

```
function minimax (node, depth, maximizingPlayer)
  if node is "End Node" or depth = 0
    return value(node)
  if maximizingPlayer
    Max = -  $\infty$ 
    foreach child of node
      Max = max(Max, minimax(child, depth-1, False))
    return Max
  else
    Min =  $\infty$ 
    foreach child of node
      Min := min(Min, minimax(child, depth-1, True))
    return Min
```

Minimax (Ví dụ)



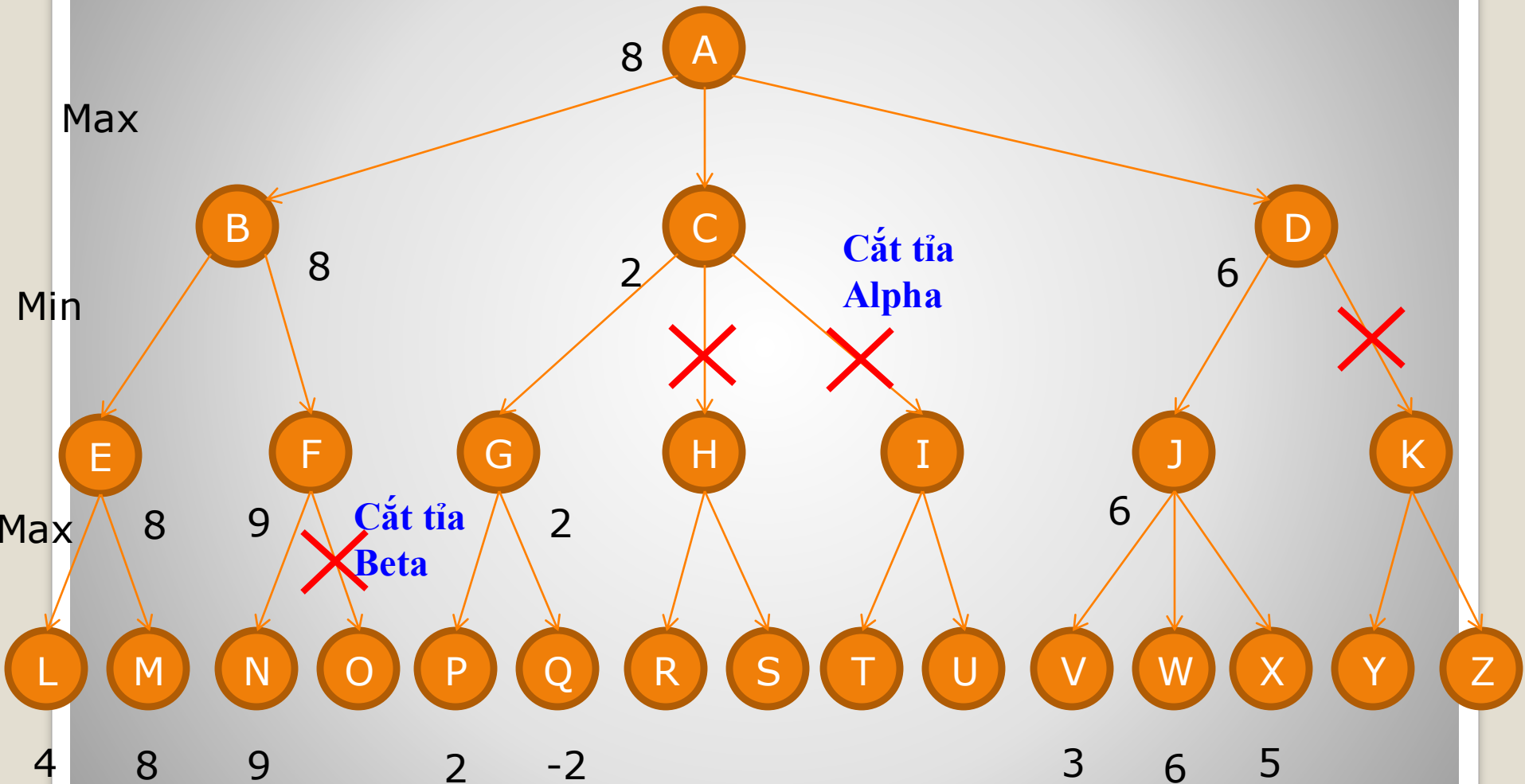
MiniMax với độ sâu giới hạn

- ✓ Trong việc tìm kiếm Minimax, cho dù giới hạn độ sâu thì số đỉnh con của một đỉnh bất kỳ vẫn rất lớn trong thực tế
- ✓ Ví dụ:
 - ✓ Cờ vua: số nhánh trung bình khoảng 35.
 - ✓ Vậy nếu cần suy nghĩ độ sâu là 4 thì phải mất ít nhất $35^4 = 1.5 * 10^6$ số phép đánh giá.
- ✓ → Cần một phương pháp **làm giảm số nhánh**

Cắt tỉa alpha-beta

- ✓ Mục đích làm giảm số nhánh trong cây tìm kiếm và,
- ✓ Không làm ảnh hưởng đến sự đánh giá của đỉnh đang xét.
- ✓ → Phương pháp cắt tỉa alpha-beta cho phép cắt bỏ các nhánh không cần thiết cho sự đánh giá của đỉnh đang xét.

Alpha-beta (Ví dụ)



Cắt tỉa Alpha beta- Thuật Toán 1

```
function minimax (node, depth, maximizingPlayer)
  return alphabeta(node, depth, -  $\infty$ ,  $\infty$ , maximizingPlayer)
```

```
function alphabeta (node, depth, a, b, maximizingPlayer)
  if node is "End Node" or depth = 0
    return value(node)
  if maximizingPlayer
    foreach child of node
      a = max(a, alphabeta (child, depth-1, a, b, False))
      if a >= b break;
    return a
  else
    foreach child of node
      b := min(b, alphabeta (child, depth-1, a, b, True))
      if a >= b break;
    return b
```

Cắt tỉa Alpha beta – Thuật Toán 2

```
function minimax (node, depth)
  return alphabeta(node, depth, -  $\infty$ ,  $\infty$ )
```

```
function alphabeta (node, depth, a, b)
  if node is "End Node" or depth = 0
    return value(node)
  foreach child of node
    a = max(a, -alphabeta (child, depth-1, -b, -a))
    if a >= b break;
  return a
```

Demo